

M8 — Notifications & Alerts

Module Specification

Trading Journal App · v1.0 · July 2026 · Status: ready to break into tickets

Field	Value
Module ID	M8
Track / owner	Intelligence & Engagement track — same owner as M7; second of four modules (M7 → M8 → M9 → M10)
Depends on	M0 Foundation & Shared Contracts · M7 AI Coach (Insight, ReviewSummary) · M4 Strategy & System Builder (RuleCheck) · M1 Identity (contact channels, timezone)
Blocks	Nothing downstream strictly requires M8 to function, but M10 (streaks and badges) reads notification engagement data as one of its gamification signals — see Section 12.
Scope in	Routing M7's insights/reviews and M4's rule-check breaches to push, email and in-app notifications; per-category channel and cadence preferences; digest assembly; device token management; quiet hours and category snoozing
Scope out	Generating insight or review content (M7) · defining a rule check (M4) · computing P&L or facts (M3) · rendering dashboards (M6) · streaks and badges (M10)

Purpose

M8 is the module that reaches out instead of waiting to be opened. Everything M7 writes and every rule breach M4 detects sits quietly in this product until M8 decides when, where, and whether to tell the trader about it. M8 has one hard rule it never breaks: it decides timing and channel, never content — it cannot alter a word of what M7 or M4 said, and it cannot write back into either module.

Components in this specification

1. Stories
2. Data flow and schema
3. UI screens
4. Detailed description and business logic
5. Flowcharts
6. Test plan
7. Quality benchmarks
8. Security constraints

9. Error handling
10. External dependencies
11. Performance and scalability
12. Module relationships
13. Data policies (EU and US)
14. Documentation

1. Stories

Fourteen stories. Eleven map directly onto the five UI screens in Section 3; US-12, US-13 and US-14 describe cross-screen delivery, integrity and privacy guarantees.

ID	Story	Acceptance criteria	Pri
US-01	As a trader, I want a single inbox of everything the app has sent me, so I don't have to hunt across push, email and the app itself.	M8-S01 lists every Notification for the trader across all channels, newest first, marked read or unread.	Must
US-02	As a trader, I want a notification to take me straight to the thing it's about.	Every row in M8-S01 opens its source record directly — an Insight in M7-S02, a ReviewSummary in M7-S03, or the relevant M4 rule check.	Must
US-03	As a trader, I want to control which categories of notification I get, and on which channel.	M8-S02 lets a trader enable or disable push, email and in-app independently per category (insight, review, risk_alert, system).	Must
US-04	As a trader, I want risk alerts to reach me immediately, not bundled into tomorrow's digest.	The risk_alert category defaults to immediate cadence and cannot be set to a digest-only cadence (Section 4.1).	Must
US-05	As a trader, I want insights and reviews delivered as a daily or weekly digest instead of one at a time.	M8-S02's cadence control for insight and review categories offers immediate, daily digest, or weekly digest (Figure 8.1).	Should
US-06	As a trader, I want to see what a digest will look like before it's sent.	M8-S04 previews the current pending DigestBatch content ahead of its scheduled send time.	Should
US-07	As a trader, I want to add or remove which devices get push notifications.	M8-S03 lists every registered DeviceToken with platform and last-seen date, with a working "Remove" action per device.	Must
US-08	As a trader, I want push notifications to stop reaching a device I've	Removing a device in M8-S03 revokes its DeviceToken and no further delivery	Must

	revoked, immediately.	attempt targets it (Figure 8.3).	
US-09	As a trader, I want quiet hours so notifications don't wake me up.	M8-S05's quiet hours setting holds immediate-cadence notifications until the window ends, except risk alerts (Section 4.1).	Should
US-10	As a trader, I want to snooze a whole category temporarily without turning it off.	M8-S05's snooze control suppresses a category for a chosen duration, then automatically resumes (Section 4.4).	Should
US-11	As a trader, I want a failed delivery to retry once instead of just silently never arriving.	A failed push delivery is retried exactly once before being marked failed (Figure 8.3).	Should
US-12	As a trader, I want a notification that reaches no enabled channel to still be visible somewhere.	A Notification with no enabled channel for its category is still recorded and visible in M8-S01's inbox (Section 4.1).	Must
US-13	As a trader, I want M8 to never change what M7 or M4 actually says — it should only decide when and how I'm told.	M8's API surface exposes no write endpoint to any M7 Insight/ReviewSummary field or M4 RuleCheck field; it reads them read-only for delivery (Section 8).	Must
US-14	As a trader, I want my device tokens and contact channels handled securely and never shared.	DeviceToken and channel contact info are scoped to the owning trader, never exposed to another trader or third party, and revoked immediately on device removal or account deletion (Section 13).	Must

Ticket-splitting note

US-01, US-02 and US-13 are the smallest shippable slice — a read-only in-app inbox that surfaces what M7 and M4 already emit, with a hard-enforced no-write boundary back into either module. Push, email, digests, device management, quiet hours and snoozing can all layer on afterward.

2. Data flow and schema

Six entities. M8 owns all of them, and none of them are the content being delivered — every notification body traces back to an Insight, ReviewSummary or RuleCheck record M8 only ever reads.

Entity	Purpose	Key fields
NotificationPreference	A trader's channel and cadence choice for one category.	id · user_id · category (insight review risk_alert system) · push_enabled · email_enabled · in_app_enabled · cadence (immediate daily_digest weekly_digest)
Notification	One notification, delivered or pending, tied to a source event.	id · user_id · category · source_type (insight review rule_check system) · source_id · title · body_preview · status (pending sent failed read) · created_at · sent_at · read_at
DeviceToken	One device registered to receive push notifications.	id · user_id · platform (ios android web) · token · created_at · last_seen_at · revoked_at
DigestBatch	One assembled group of notifications sent together.	id · user_id · cadence (daily weekly) · period_start · period_end · notification_ids[] · sent_at
DeliveryAttempt	One attempt to deliver a Notification on one channel.	id · notification_id · channel · status (success failed bounced) · attempted_at · error_code
Snooze	A temporary pause on one notification category.	id · user_id · category · snoozed_until

2.1 How data moves

Routing. An event from M7 (insight.generated, review.generated) or M4 (rule_check.breached) creates a Notification, checked against the trader's NotificationPreference for that category to decide channel and timing (Figure 8.1) — the notification's title and body_preview are read from the source event verbatim, never rewritten by M8.

Digesting. Non-immediate notifications queue until their cadence's scheduled trigger, at which point they're assembled into a DigestBatch and delivered together (Figure 8.2).

Delivering. Each delivery attempt on each enabled channel is recorded as a DeliveryAttempt; a failed push attempt is retried once, and a token that proves invalid is revoked automatically (Figure 8.3).

Reading elsewhere. M8 is a near-terminal node — only M10 (streaks and badges) reads from it, and only notification engagement metadata (opened, read counts), never notification content itself (Section 12).

The one rule that keeps this clean

M8 decides timing and channel, never content. A Notification's title and body_preview are copied from the source Insight, ReviewSummary or RuleCheck at creation time — M8 has no code path that composes or edits that text, only ones that decide when and where to send it.

3. UI screens

Five screens, built as a working mockup in M8_UI_Mockups.html — open it in any browser.

ID	Screen	States built
M8-S01	Notification inbox	Empty · Has notifications
M8-S02	Notification preferences	Default · Saved
M8-S03	Push & device management	No devices · Has devices
M8-S04	Digest preview	Daily · Weekly
M8-S05	Quiet hours & snooze	Default · Saved

3.1 Rules that apply to every screen

- Trader-owned, trader-facing. All five screens belong to the trader's own app, gated by ordinary account auth (Section 8) — there is no staff-only surface in M8.
- Risk alerts are visibly different. Wherever risk_alert settings appear (M8-S02, M8-S05), the UI shows the override rather than silently ignoring an attempt to weaken it.
- Nothing here writes to another module's data. M8 can create a Notification, DeviceToken, DigestBatch, DeliveryAttempt, NotificationPreference or Snooze, and nothing else.
- An empty state explains what's coming, not just that nothing's there yet — M8-S01 and M8-S03 both need this for a genuinely new account.

4. Detailed description and business logic

The rules below are the module's real content. Section 4.1's risk-alert override is the one every other rule in this module bends around.

4.1 Channels, cadence, and the risk-alert override

Rule	Decision
Every category has independent channel and cadence controls	Push, email and in-app are enabled or disabled per category, not globally — a trader can, for instance, get reviews by email only and insights in-app only.
Risk alerts cannot be digested	The risk_alert category's cadence is locked to immediate — there is no digest-only option in the UI, and a direct API attempt to set one is rejected (PRF-001).
Risk alerts cannot be fully silenced	At least one channel must remain enabled for risk_alert at all times; disabling the last one is rejected the same way (PRF-001).

A notification with no enabled channel is still recorded	If every channel for a category is off, the Notification row is still created and appears in the inbox (M8-S01) — “no channel enabled” is a delivery decision, not a reason to lose the notification entirely (US-12).
--	--

4.2 Digest assembly

Rule	Decision
Only insight and review categories digest	risk_alert and system notifications are never included in a DigestBatch, regardless of a trader's cadence setting for other categories.
A digest is built once per cadence trigger	Daily and weekly digest assembly each run on their own fixed schedule (Figure 8.2); opening M8-S04 reads the current queue, it never triggers assembly.
An empty period sends nothing	If no notifications are queued when a digest's cadence triggers, no DigestBatch is created and nothing is sent — an empty digest email is treated as a failure to avoid, not a valid low-content one (Figure 8.2).

4.3 Device token lifecycle

Rule	Decision
Tokens are registered per device, not per account	A trader can have any number of active DeviceTokens across phones, browsers and tablets; push delivery attempts every currently active one for that trader.
An invalid token is revoked automatically	If a delivery attempt reports the token as invalid or expired, it is revoked immediately rather than retried indefinitely against a dead endpoint (Figure 8.3).
Manual removal and automatic revocation use the same path	Removing a device from M8-S03 and an automatic invalid-token revocation both set DeviceToken.revoked_at — there is no separate, weaker manual-removal state.

4.4 Quiet hours and snooze

Rule	Decision
Quiet hours hold, they don't drop	An immediate-cadence notification generated during quiet hours is held and delivered at the window's end, not discarded — except risk_alert, which is exempt entirely.
Snooze is category-scoped and always temporary	Snoozing a category (M8-S05) suppresses its notifications until a chosen end date; there is no permanent snooze in

	v1, mirroring M7's pattern-mute design (M7, Section 4.3).
Risk alerts are exempt from both	Quiet hours and snooze settings have no effect on <code>risk_alert</code> — the same override principle as Section 4.1, applied consistently across the module.

4.5 Delivery and retry

Rule	Decision
Every attempt is recorded, not just the outcome	A <code>DeliveryAttempt</code> row exists for each channel each time delivery is tried, so a “why didn't I get this” investigation has a full trail, not just a final status.
Exactly one retry, then stop	A failed delivery is retried once; if the retry also fails, the <code>Notification</code> is marked failed and no further automatic attempt is made on that channel (Figure 8.3).
A failed channel doesn't block other channels	If push fails but email succeeds for the same <code>Notification</code> , the notification is still considered delivered overall — each channel's <code>DeliveryAttempt</code> is independent.

4.6 Read state and inbox retention

Rule	Decision
Read state is per-notification, tracked once	Opening a notification's source from M8-S01 marks it read; there is no separate per-channel read state to reconcile.
The inbox is a durable record, not a transient log	A <code>Notification</code> stays visible in M8-S01 after being read — it is not removed or auto-archived, so a trader can always scroll back through what the app has told them.

5. Flowcharts

Three diagrams: how a source event becomes a routed notification, how a digest batch is built and sent, and how a push delivery is retried or a dead token is revoked. Each is followed by the reasoning behind its branches.

5.1 From a source event to a delivered notification

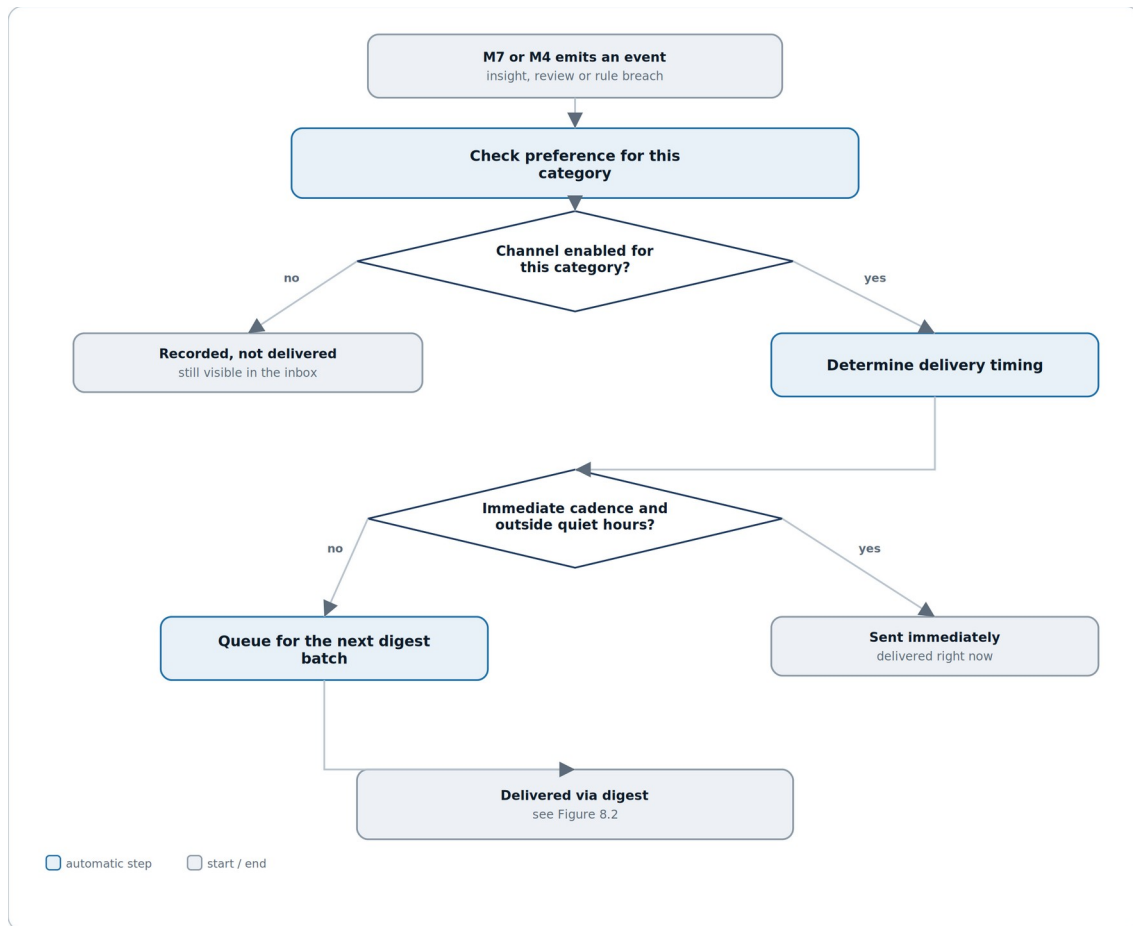


Figure 8.1 — a notification is recorded regardless of channel, but timing depends on cadence and quiet hours.

Reading it. The first gate is simply whether any channel is enabled at all for the category — a “no” still records the notification, it just never attempts delivery. The second gate, reached only after at least one channel is confirmed enabled, decides between sending right now and queuing for the next digest.

Why recording happens before the channel check, not after. A trader who later re-enables a category should be able to see what they missed while it was off — that's only possible if the Notification row exists regardless of whether delivery was attempted (US-12).

5.2 Building and sending a digest batch

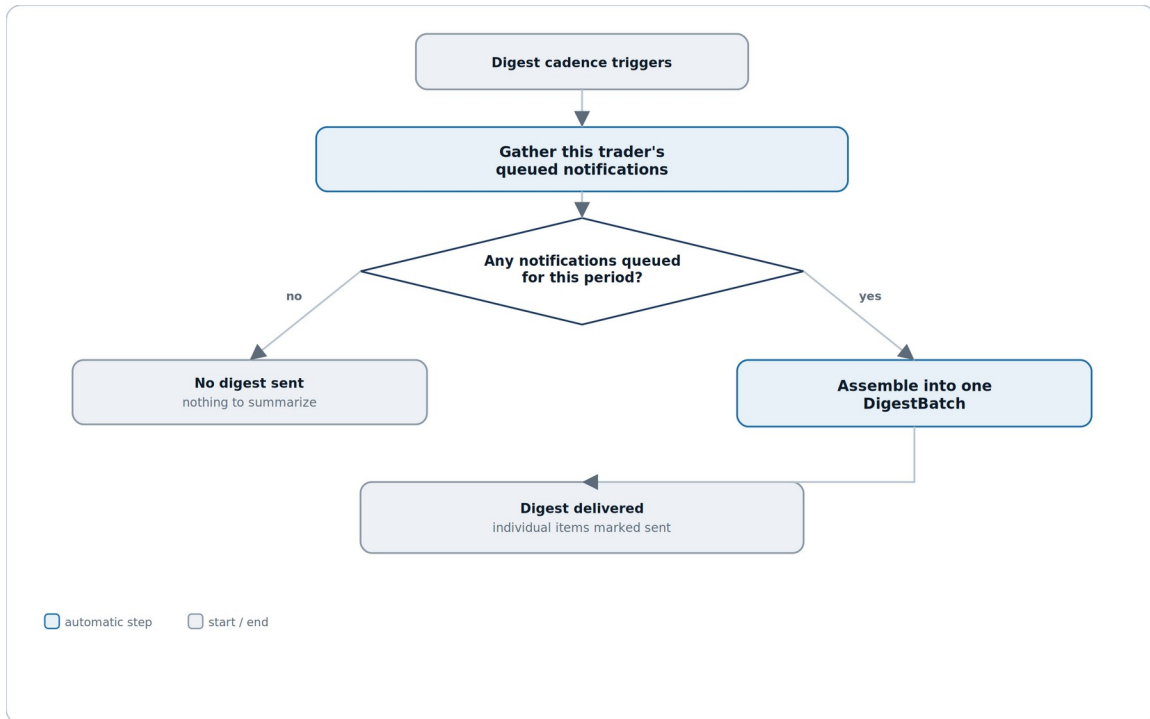


Figure 8.2 — an empty queue produces no digest at all, not an empty one.

Reading it. This diagram is deliberately simple: one trigger, one check, two outcomes. There's no retry or partial-send logic here because a digest is additive by nature — anything that doesn't make this cycle's batch simply waits for the next one instead of failing.

Why an empty period sends nothing rather than a “you have no updates” email. A digest with nothing in it trains a trader to stop opening it — silence is the more honest and more respected signal when there genuinely isn't anything to report.

5.3 Delivering a push notification and handling failure

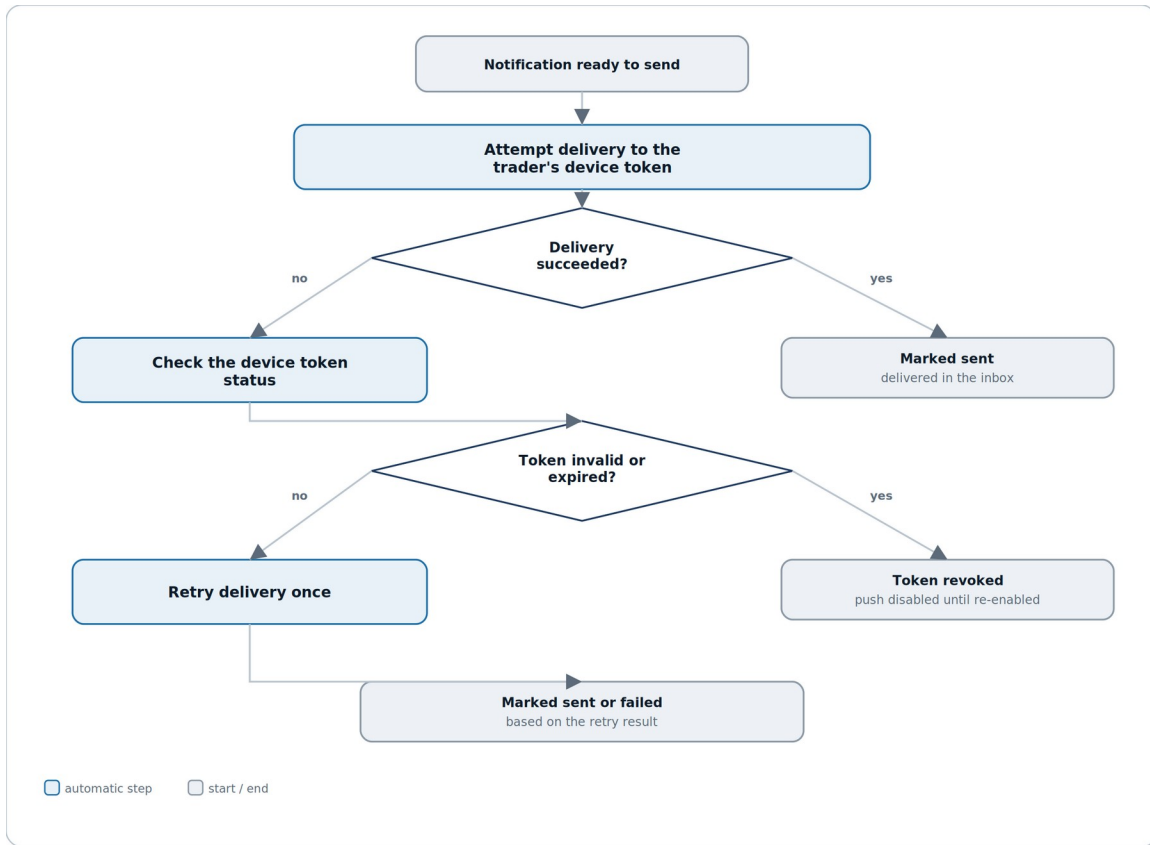


Figure 8.3 — a dead token gets revoked automatically; a live one gets exactly one retry.

Reading it. The two diamonds ask different questions in sequence: first, did this specific attempt succeed; second, only if it didn't, is the token itself the problem. That ordering means a temporary network blip and a genuinely dead token are never treated the same way.

Why token revocation and manual device removal converge on the same end state. Whether a trader removes a device themselves (M8-S03) or the system discovers the token is dead, the result is identical — a revoked token that no future delivery attempt will target — so there's exactly one way to reason about “is this device still receiving pushes,” not two.

6. Test plan

The riskiest failure mode in this module is a silent one: a risk alert that gets caught by a digest, a quiet-hours window, or a disabled channel it should have been exempt from — the kind of bug that only matters on the one day a trader actually needed the alert.

Layer	What it covers	Key cases
Unit	Pure logic with no I/O.	Channel/cadence resolution given a fixture NotificationPreference · quiet-hours window math across a day boundary · retry/backoff logic for a single DeliveryAttempt
Integration	The module against its own store and its data sources.	A Notification's title/body_preview exactly matches its source Insight/ReviewSummary/RuleCheck text at creation time · a DigestBatch's notification_ids match the queue at assembly time · a revoked DeviceToken never appears in a subsequent delivery attempt
Contract	What M8 promises other modules.	M8 issues zero write calls to M7 or M4's mutation endpoints under any user action — verified by call-count assertion
Override regression	The risk-alert exemption.	0 instances, across the full regression suite, of a risk_alert notification being digested, held by quiet hours, snoozed, or fully silenced
Fidelity regression	The no-content-alteration rule.	0 instances of a Notification's stored title or body_preview diverging from its source record's text beyond approved truncation
Delivery	Retry and revocation behaviour.	A simulated invalid-token response triggers exactly one revocation and zero further attempts against that token · a simulated transient failure triggers exactly one retry, not more

6.1 Manual QA checklist

#	Manual QA checklist item
1	A risk alert fires and is delivered immediately even with quiet hours active and the insight category snoozed.

2	Disabling every channel for a non-risk category still leaves its notifications visible in M8-S01.
3	Attempting to disable the last enabled channel for risk_alert in M8-S02 is blocked with a clear message.
4	A daily digest preview in M8-S04 matches exactly what's actually delivered at the scheduled send time.
5	Removing a device in M8-S03 stops a subsequent test push from reaching it, with no further attempt logged.
6	A notification's text in M8-S01 matches the source Insight or ReviewSummary's text word for word.
7	Snoothing a category in M8-S05, then waiting past its expiry, automatically resumes notifications for that category.
8	No manual testing path produces a network call to M7's insight-generation or M4's rule-definition endpoints.

6.2 Test data

- A seeded account with at least one active DeviceToken and a history of both successful and failed delivery attempts.
- A trader with every channel disabled for one non-risk category, to verify the still-recorded-in-inbox behaviour.
- A trader with quiet hours enabled and a pending risk alert, to verify the exemption directly.
- A queue of several pending notifications spanning a scheduled digest boundary, to test assembly timing.
- A fixture set of simulated delivery failures (transient and permanent/invalid-token) for the retry and revocation test layer.

7. Quality benchmarks

Numbers, not adjectives. Anything unmeasurable is not a benchmark.

Metric	Target	How it is measured
Inbox load time	M8-S01 fully rendered within 1 s, p95	Time from navigation to last notification row painted, on a seeded account with a year of history
Push delivery latency	An immediate-cadence push is attempted within 5 s, p95, of the source event	Server timing from source event received to delivery attempt initiated
Digest assembly completion	Scheduled digest batches complete within their window with 0 dropped traders	Batch job completion monitoring
Delivery success rate	At least 98% of attempted deliveries succeed on first attempt or the single retry, rolling 30-day window	Aggregated DeliveryAttempt outcomes
Risk alert delivery reliability	0 risk_alert notifications lost, delayed beyond the immediate window, or incorrectly digested/held/snoozed, across the full regression suite	Automated fixture-based delivery-tracking assertion
Content fidelity	0 instances of a notification's body diverging from its source Insight, ReviewSummary or RuleCheck text beyond approved truncation	Automated text-comparison assertion (Section 6)

Why risk alert delivery reliability is the number that matters most

Every other benchmark here is about speed or fidelity. This one is about the one category of notification in this entire product where a delay or a silent drop has real consequences outside the app — a trader who doesn't find out their rule was breached until the next digest has lost the entire point of the alert. Section 4.1's override exists specifically to make this failure structurally hard to cause; this benchmark exists to prove it stays that way.

8. Security constraints

M8 handles two things that need particular care: device identifiers capable of reaching a trader outside the app, and a read-only relationship with M7 and M4 that must never become a write path.

Area	Constraint
Ownership scoping	Every NotificationPreference, Notification, DeviceToken, DigestBatch, DeliveryAttempt and Snooze is scoped to the

	requesting trader's own user_id — no cross-trader read path exists anywhere in M8.
Read-only enforcement toward M7 and M4	M8's API surface exposes no write endpoint to any M7 or M4 table — enforced at the service boundary, so nothing in M8 can alter an Insight, ReviewSummary or RuleCheck.
Device tokens are opaque and revocable	A DeviceToken value is never exposed back to the client beyond what's needed to display platform/last-seen; revocation is immediate and irreversible for that token.
Notification previews avoid full sensitive content	body_preview is a truncated teaser, never the full text of a journal entry or chat message — this limits exposure on a lock-screen push banner even if the device itself is otherwise secure.
Delivery provider credentials are never trader-facing	Push and email provider API keys are held only in the delivery service, never passed to or through any client-facing surface.

9. Error handling

Every error has a code for logs, a defined system behaviour, and what the trader actually sees.

Code	Scenario	System behaviour	What the trader sees
DEV-001	Device token registration fails validation	Registration rejected	"Couldn't enable push on this device — try again"
DEV-002	A push delivery attempt returns an invalid-token response from the provider	Token auto-revoked (Figure 8.3)	No push arrives on that device; a later visit to M8-S03 shows it no longer listed as active
DIG-001	Digest assembly finds zero queued notifications for a period	No DigestBatch created; not an error	No digest arrives — no visible change
DLV-001	A delivery attempt fails and the single retry also fails	Notification marked failed for that channel	The notification remains visible in M8-S01 regardless of the failed channel
SNZ-001	A snooze duration exceeds the maximum allowed window	Save blocked	"Choose 30 days or less"
PRF-001	A preference save would leave risk_alert with zero channels enabled, or a digest-only cadence	Save blocked (Section 4.1)	"Risk alerts can't be fully disabled or digested — choose at least one immediate channel"
QH-001	Quiet hours start and end times are identical or otherwise invalid	Save blocked	"Choose a valid quiet hours window"

Why DIG-001 is explicitly not treated as an error

An empty digest period is the expected, common case for a trader with a quiet week — logging it as an error would create noise that trains whoever's on call to ignore digest-related alerts entirely, which is exactly the wrong habit to build in a module that also handles risk alerts.

10. External dependencies

M8 is the first module in this product with a genuine external delivery dependency — push and email providers it does not control the uptime of.

Dependency	Used for	If it fails	Fallback
M7 AI Coach	Insight and ReviewSummary generation events as notification sources	No new insight/review notifications are created	Existing notifications remain visible in M8-S01; risk alerts from M4 are entirely unaffected
M4 Strategy & System Builder	RuleCheck breach events as risk alert sources	No new risk alert notifications are created	This is the one dependency failure worth escalating immediately, given Section 7's flagship guarantee
M1 Identity	Contact channels (email address) and timezone for quiet hours resolution	Email delivery and quiet-hours timing are unavailable	Push and in-app delivery continue unaffected; quiet hours falls back to UTC boundaries until resolved
Push notification provider	Delivering push notifications to registered DeviceTokens	Push delivery fails (DLV-001)	Email and in-app delivery continue independently per channel (Section 4.5)
Email delivery provider	Delivering email notifications and digests	Email delivery fails (DLV-001)	Push and in-app delivery continue independently per channel
M0 Foundation	Shared data model, design system, error contract	N/A — foundational	N/A

Why M4's dependency gets called out specially

Every other dependency in this table degrades gracefully — the trader sees less, not something wrong. A failure in the M4 event feed is different: it means risk alerts silently stop being generated at the source, which Section 7 treats as the single failure this module cares most about catching fast.

11. Performance and scalability

Concern	Position
Scale assumption for v1	Up to a few thousand notifications per trader per year, and up to 10 active DeviceTokens per trader — both naturally small and slow-growing.
Hottest path	Immediate-cadence delivery of a risk alert — the one path

	where latency has real consequences (Section 7).
Digest assembly is inherently batchable	Daily and weekly digest jobs process the full trader base on a fixed schedule and can be parallelized freely; nothing about digest assembly requires sub-second response time.
Delivery fan-out is bounded per trader	A single notification fans out to at most a handful of channels and devices — there is no execution path where one event triggers an unbounded number of delivery attempts.
What breaks first	Push provider throughput during a broad market event that triggers many traders' risk alerts simultaneously — the reason DLV-001's single-retry design exists rather than an unbounded retry loop that would compound the load.
Growth triggers	Revisit design if push delivery latency (Section 7) degrades under simultaneous-alert load, or if per-trader DeviceToken counts regularly exceed the v1 assumption.

12. Module relationships

M8 sits downstream of M7 and M4 as a pure delivery layer — it reads events from both and exposes almost nothing back, the same discipline M6 and M7 established before it.

Direction	Module	What crosses the boundary
Consumes	M7 AI Coach	insight.generated and review.generated events, read-only, as notification content sources.
Consumes	M4 Strategy & System Builder	rule_check.breached events, read-only, as risk alert sources.
Consumes	M1 Identity	The trader's contact channels (email) and timezone, for delivery and quiet-hours resolution.
Consumes	M0 Foundation	Shared data model, design system, and error contract.
Exposes	M10 Streaks & Badges	Notification engagement metadata only (opened, read counts) — never notification content, which stays inside M8.
Emits	—	notification.sent · notification.read · digest.sent · device.revoked (for the trader's own audit history, and M10's engagement signal only)

The dependency to watch

M8 is the second module (after M7) whose output M10 reads — keep that surface exactly as narrow as Section 12 states: engagement counts only, same principle M7 established. The other direction matters just as much: M8 must never become a path where a delivery-timing decision quietly turns into a content edit — if a trader ever sees a notification that doesn't match its source, that's a Section 4 violation, not a formatting bug.

13. Data policies (EU and US)

M8 holds two things worth separating clearly: routing metadata about a trader's own content (low sensitivity on its own), and device identifiers plus delivery records that, in aggregate, describe a trader's device and usage patterns.

13.1 What we hold and why

Data	Classification	Lawful basis (GDPR)
NotificationPreference, Snooze	Personal data — low sensitivity, a trader's own delivery settings	Contract — the notification feature the trader opted into
Notification	Personal data — a routing record referencing content that itself may be sensitive (Insight/ReviewSummary text, truncated)	Contract
DeviceToken	Personal data — a device identifier capable of directly reaching the trader outside the app	Contract; also legitimate interest in preventing delivery to a device the trader no longer controls
DigestBatch, DeliveryAttempt	Personal data — operational delivery records, low sensitivity on their own	Contract; legitimate interest in maintaining delivery reliability and auditability

13.2 GDPR (EU / UK)

Obligation	How M8 meets it
Lawful basis (Art. 6)	Contract for preference and notification data; legitimate interest, balanced against the trader's own control via M8-S03, for device and delivery records.
Data minimisation (Art. 5(1)(c))	Notification.body_preview is a truncated teaser, not a full copy of the source content — the full text is always read live from M7 at the point a trader opens it, not duplicated here.
Right of access / portability (Art. 15/20)	Notification history and preferences are included in M1's account-wide export; DeviceToken values themselves are excluded from export as pure technical identifiers with no standalone meaning to the trader.
Erasure (Art. 17)	On user.deletion_requested from M1, M8 must erase every NotificationPreference, Notification, DeviceToken, DigestBatch, DeliveryAttempt and Snooze row for that user, revoking any active DeviceToken immediately as part of the cascade.
Automated decision-making (Art. 22)	M8 makes delivery-timing decisions (channel, cadence, quiet hours) but these have no legal or similarly significant

	effect on the trader — they govern notification convenience, not any determination about them.
Security of processing (Art. 32)	Ownership scoping, read-only enforcement toward M7/M4, and opaque non-exportable device tokens (Section 8) protect both M8's own tables and the routing metadata it holds.

13.3 United States (CCPA / CPRA and state law)

Requirement	How M8 meets it
Right to know / access	Same account-wide export as GDPR access, via M1, excluding raw DeviceToken values for the same reason.
Right to delete	Same erasure flow as GDPR Art. 17, propagated within the account-level grace period, including immediate device revocation.
Sale or sharing of personal information	M8 does not sell or share device identifiers, delivery records, or notification content with any third party; push and email providers act strictly as processors delivering on the trader's behalf.

13.4 Retention schedule

Data	Retention	Reason
NotificationPreference, Snooze	Life of account + 30-day grace (matches M1-M7)	Low-cost settings expected to persist for the life of the account
Notification, DigestBatch	Life of account + 30-day grace	The inbox is a durable record by design (Section 4.6), not a transient log
DeviceToken	Deleted immediately on revocation, or life of account + 30-day grace if never revoked	An active token has no reason to outlive its usefulness; a revoked one has no reason to be retained at all
DeliveryAttempt	180 days from attempt, then auto-deleted	Operational/audit data useful for near-term troubleshooting, not a record a trader needs to keep long-term

The one thing not to get wrong

A DeviceToken must be usable for delivery only while it's valid and unrevoked — the moment a device is removed in M8-S03, an account is deleted, or a token is flagged invalid by the provider, that token must stop being a live delivery target immediately, not on the next batch cycle. A push notification reaching a device the trader believes they've disconnected is the kind of failure that undermines trust in every other control on this page.

14. Documentation

Documentation is part of the module, not an afterthought once the code works.

Deliverable	Contents
Module README	What M8 does, the risk-alert override in Section 4.1 in full, and how routing, digesting and delivery are operationally separated.
Contract / API reference	Every field on NotificationPreference, Notification, DeviceToken, DigestBatch, DeliveryAttempt and Snooze, and the exact read contract M8 uses against M7 and M4, generated from the agreement in Section 12 and M0.
Decision log (ADRs)	Initial entries: why risk alerts are structurally exempt from digesting/quiet hours/snoozing rather than relying on a default setting; why exactly one retry rather than exponential backoff; why M10 only ever sees engagement counts, never notification content.
Runbook	What to do when a push or email provider has an outage, how to investigate a trader-reported “I didn't get notified” ticket by tracing a Notification through its DeliveryAttempts, and how to force-revoke a device token outside the normal flow.
Data inventory & retention register	The tables in Section 13, maintained as living documents, with DeviceToken's immediate-revocation handling called out explicitly.
UI mockups	M8_UI_Mockups.html — all five screens with every state.

14.1 Definition of done

M8 is done when all of the following are true:

- Every story meets its acceptance criteria.
- The risk-alert override in Section 4.1 is verified against live M4 rule-check events, not just mocks.
- All five screens are built with every state, including the locked risk-alert controls and the empty device list.
- Every error in Section 9 is reachable and produces the described system behaviour.
- The risk alert delivery reliability and content fidelity guarantees (Section 7) pass their full automated regression suite.
- Benchmarks in Section 7 are measured against a realistic simulated delivery load, not assumed.
- README, ADRs, runbook are written; the data inventory is filled in with DeviceToken's revocation handling called out.
- Deletion propagation is verified end to end from M1 through to all six of M8's own tables, including immediate device revocation.
- Demoed to the other module owners, with the risk-alert override walked through explicitly — this is the module's one rule that cannot ever have an exception.

End of M8 specification — v1.0. Same 14-component structure as M1-M7 — the second module in the Intelligence & Engagement track, with M9 next.